



Technical Report

SECJ3623: Mobile Application Programming
Semester I, Session 2025/2026

Project Title: Smart Recipe Finder
Group Name: NULL

BIL	Name	Matric No
1	Mohammed Satippiddin Safid Bin Hamzah	SX210295ECJHS01
2	Mohamad Roslan Bin Saidi	SX180266CSJS04
3	Nurul Aidatul Aisyqin	SX211954ECJHF04
4	Muhammad Asyraf Bin Ramlee	SX240243ECJHS04

Table of Contents

1. Development Approach and Workflows	3
1.1 Methodology	3
1.2 Workflows Stages	4
1.2 Directory Structures	5
2. Software, Tools, and Frameworks	7
2.1 Core Technologies	7
2.2 Key Dependencies	7
2.3 Development Tools.....	8
3. Database & Data Handling Design	8
3.1 Data Model	8
3.2 Handling CRUD Operation	10
5. Important Source Code Snippets with Explanations	13
5.1 Main Application Initialization.....	13
5.2 Firebase connection	15
6. Testing Strategy	16
6.1 Unit Testing (Logic Validation).....	16
6.2 Widget Testing (UI Component Verification)	16
6.3 Integration Testing (End-to-End User Flow)	17

6.4 Manual Testing (Device and Usability Testing).....	17
6.5 Security and Role-Based Access Testing (Firebase Rules Validation)	17
6.7 Performance and Stability Testing.....	18
7. Technical Challenges and Solutions.....	18
7.1 Guest Access With Restricted Features	18
7.2 Firestore Data Structure and Query Limitations.....	19
7.3 Image Upload and Retrieval Handling	19
7.4 Navigation and Screen Access Control	19
7.5 Offline and Weak Network Behavior	20
8. Conclusion	21

1. Development Approach and Workflows

1.1 Methodology

This project was developed using an Agile-based approach, where the development process was divided into small, manageable phases. Features were built incrementally and tested regularly to support flexibility and quick changes based on user needs and feedback.

Smart Recipe Finder was developed based on the following principles:

- **Modular Architecture:** The application is organized into three main layers:
 - **Service Layer:** Handles Firebase operations and business logic (recipe retrieval, favorites, upload, sharing).
 - **Model Layer:** Defines data structures such as Recipe, User, and Favorite/Bookmarks.
 - **UI Layer:** Displays content through screens and reusable widgets (recipe list, recipe details, search, add recipe).
- **Cloud-First Development:** All recipes, images, and user data are stored in Firebase instead of local storage. This ensures data availability across devices and simplifies centralized data management.
- **User-Friendly Access (Guest-First):** Browsing and searching recipes are available without login to support casual users, while login is only required for management functions such as saving favorites, uploading recipes, and sharing with identity.
- **Cross-Platform Support:** The application is built using Flutter to support multiple platforms (Android and iOS) without rewriting the codebase.

1.2 Workflows Stages

Phase 1: Project Initialization & Setup

- Created the Flutter project and folder structure
- Connected the project to Firebase
- Configured Firestore for recipe data storage
- Configured Firebase Storage for recipe images
- Implemented a basic theme and navigation structure

Phase 2: Core Infrastructure Development

- Designed Firestore database structure (recipes, users, favorites/bookmarks)
- Developed data models (Recipe, User, Favorite)
- Built service classes to handle CRUD operations with Firebase
- Set up basic state management for data fetching and UI updates

Phase 3: Public Browsing and Search Features (No Login)

- Implemented recipe listing for guests
- Implemented search by keyword (title/ingredients)
- Implemented filters (favorites, recipe creator, preparation time)
- Implemented recipe details view (ingredients, steps, image)

Phase 4: User Features (Login-Based Management)

- Implemented authentication (Firebase Auth) for registered users
- Implemented favorites/bookmark feature

- Implemented add/edit/delete recipe (user-owned recipes)
- Implemented sharing feature (share recipe info through device sharing)

Phase 5: Testing & Optimization

- Conducted functional testing for browse, search, and view flows
- Tested authentication and permission-based access (guest vs registered user)
- Verified Firebase security rules (read-only for guests, write control for owners)
- Improved UI responsiveness and performance (loading states, caching where suitable)
- Fixed bugs and ensured stability for release readiness

1.2 Directory Structures

The application follows a layered architecture:

```
lib/
├── app/
│   ├── models/
│   │   ├── recipe.dart
│   │   └── user.dart
│   ├── screens/
│   │   ├── auth/
│   │   ├── bookmarks/
│   │   │   └── bookmarks_screen.dart
│   │   ├── home/
│   │   ├── manage/
│   │   │   └── manage_screen.dart
```

```

| | | |— recipe_form_dialog.dart
| | | |— user_form_dialog.dart
| | | |— onboarding/
| | | |— onboarding_screen.dart
| | | |— profile/
| | | |— recipes/
| | | |— recipe_detail_screen.dart
| | | |— recipes_screen.dart
| | | |— splash/
| | |— services/
| | |— auth_service.dart
| | |— bookmark_service.dart
| | |— manage_recipe_service.dart
| | |— manage_user_service.dart
| | |— recently_viewed_service.dart
| | |— recipe_service.dart
| | |— theme/
| | |— app_theme.dart
| | |— widgets/
| | |— bottom_nav.dart
| | |— shell.dart
|— firebase_options.dart
— main.dart

```

2. Software, Tools, and Frameworks

2.1 Core Technologies

Technology Stack	Version	Purpose
Flutter	Latest Stable	Cross-platform mobile framework
Dart	3.3.0	Programming languages
Firebase	Latest	Database and backend services

2.2 Key Dependencies

```
name: recipe_finder
description: A Recipe Finder Application

version: 1.0.0+1

environment:
  sdk: '>=3.3.0 <4.0.0'

dependencies:
  flutter:
    sdk: flutter

  firebase_core: ^4.3.0
  firebase_auth: ^6.1.3
  cloud_firestore: ^6.1.1

  provider: ^6.1.2
  google_fonts: ^6.2.1
  carousel_slider: ^5.0.0
  flutter_spinkit: ^5.2.0
  cached_network_image: ^3.4.1
  shared_preferences: ^2.2.2
  share_plus: ^12.0.0

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_launcher_icons: ^0.13.1

flutter_launcher_icons:
  android: true
  ios: true
  image_path: assets/icons/app_icon.png

flutter:
  uses-material-design: true

assets:
```



```
- assets/images/  
- assets/images/onboarding/  
- assets/images/home/  
- assets/images/recipes/
```

2.3 Development Tools

- IDE: Visual Studio Code
- Emulator Engine: Xcode iOS and Android Studio
- Firebase Console: Backend Management
- Flutter DevTools

3. Database & Data Handling Design

3.1 Data Model

Recipe Model

```
class Recipe {  
  final String id;  
  final String ownerId;  
  final String title;  
  final String description;  
  final String category;  
  final int totalTimeMinutes;  
  final List<String> ingredients;  
  final List<String> howToCook;  
  final List<String> photoUrls;  
  final bool isDeleted;  
  final int savedCount;  
  
  const Recipe({  
    required this.id,  
    required this.ownerId,  
    required this.title,  
    required this.description,  
    required this.category,  
    required this.totalTimeMinutes,  
    required this.ingredients,  
  })
```

```
required this.howToCook,  
required this.photoUrls,  
required this.isDeleted,  
required this.savedCount,  
});
```

User Model

```
class AppUser {
    final String id;
    final String email;
    final String name;
    final String role;
    final DateTime createdAt;
    final DateTime updatedAt;

    AppUser({
        required this.id,
        required this.email,
        required this.name,
        required this.role,
        required this.createdAt,
        required this.updatedAt,
    });
}
```

3.2 Handling CRUD Operation

Create Recipe

```
Future<void> createRecipe({
    String? ownerId,
    required String title,
    required String description,
    required String category,
    required int totalTimeMinutes,
    required List<String> ingredients,
    required List<String> howToCook,
    required List<String> photoUrls,
}) async {
    final uid = ownerId ?? FirebaseAuth.instance.currentUser?.uid;
    if (uid == null) {
        throw Exception("Not signed in");
    }

    final now = FieldValue.serverTimestamp();

    await _recipes.add({
        'ownerId': uid,
        'title': title.trim(),
        'description': description.trim(),
        'category': category.trim(),
        'totalTimeMinutes': totalTimeMinutes,
        'ingredients': ingredients,
        'howToCook': howToCook,
        'photoUrls': photoUrls,
        'savedCount': 0,
        'isDeleted': false,
        'createdAt': now,
        'updatedAt': now,
    });
}
```

Update Recipe

```
Future<void> updateRecipe({
    required String recipeId,
    required String title,
    required String description,
    required String category,
    required int totalTimeMinutes,
    required List<String> ingredients,
    required List<String> howToCook,
    required List<String> photoUrls,
}) async {
    await _recipes.doc(recipeId).update({
        'title': title.trim(),
        'description': description.trim(),
        'category': category.trim(),
        'totalTimeMinutes': totalTimeMinutes,
        'ingredients': ingredients,
        'howToCook': howToCook,
        'photoUrls': photoUrls,
        'updatedAt': FieldValue.serverTimestamp(),
    });
}
```

Show Recipe (All and Specific by owner)

```
CollectionReference<Map<String, dynamic>> get _recipes =>
    _db.collection('recipes');

Stream<QuerySnapshot<Map<String, dynamic>>> streamMyRecipes(String uid) {
    return _recipes
        .where('ownerId', isEqualTo: uid)
        .where('isDeleted', isEqualTo: false)
        .orderBy('createdAt', descending: true)
        .snapshots();
}
```

Delete Recipe (Soft Delete)

```
Future<void> softDeleteRecipe(String recipeId) async {
    await _recipes.doc(recipeId).update({
        'isDeleted': true,
        'updatedAt': FieldValue.serverTimestamp(),
    });
}
```

Authentication Register, Login, Logout and Forgot Password Request

```
Future<String?> signUp({
  required String name,
  required String email,
  required String password,
}) async {
  try {
    final userCred = await _auth.createUserWithEmailAndPassword(
      email: email.trim(),
      password: password.trim(),
    );

    final uid = userCred.user!.uid;

    final newUser = AppUser(
      id: uid,
      name: name.trim(),
      email: email.trim(),
      role: 'user',
      createdAt: DateTime.now(),
      updatedAt: DateTime.now(),
    );

    await _db.collection('users').doc(uid).set({
      ...newUser.toMap(),
      'isBlocked': false,
      'photoUrl': '',
    });

    return null;
  } on FirebaseAuthException catch (e) {
    return e.message ?? "Sign up failed";
  } catch (e) {
    return e.toString();
  }
}

Future<String?> signIn({
  required String email,
  required String password,
}) async {
  try {
    await _auth.signInWithEmailAndPassword(
      email: email.trim(),
      password: password.trim(),
    );
    return null;
  } on FirebaseAuthException catch (e) {
    return e.message ?? "Login failed";
  } catch (e) {
    return e.toString();
  }
}

Future<void> signOut() async {
  await _auth.signOut();
}

Future<String?> sendForgotPasswordLink({required String email}) async {
  try {
    await _auth.sendPasswordResetEmail(email: email.trim());
    return null;
  } on FirebaseAuthException catch (e) {
    return e.message ?? "Failed to send reset link";
  } catch (e) {
    return e.toString();
  }
}
```

```

Future<(String? email, String? error)> verifyResetCode({
  required String oobCode,
}) async {
  try {
    final email = await _auth.verifyPasswordResetCode(oobCode);
    return (email, null);
  } on FirebaseAuthException catch (e) {
    return (null, e.message ?? "Invalid/expired reset code");
  } catch (e) {
    return (null, e.toString());
  }
}

Future<String?> confirmResetPassword({
  required String oobCode,
  required String newPassword,
}) async {
  try {
    await _auth.confirmPasswordReset(
      code: oobCode,
      newPassword: newPassword.trim(),
    );
    return null;
  } on FirebaseAuthException catch (e) {
    return e.message ?? "Failed to reset password";
  } catch (e) {
    return e.toString();
  }
}

```

5. Important Source Code Snippets with Explanations

5.1 Main Application Initialization

```

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const RecipeFinderApp());
}

class RecipeFinderApp extends StatelessWidget {
  const RecipeFinderApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: AppTheme.lightTheme,
      darkTheme: AppTheme.darkTheme,
      initialRoute: '/',
      routes: {
        '/': (context) => const SplashScreen(),
        '/onboarding': (context) => const OnboardingScreen(),
        '/signin': (context) => const SignInScreen(),
        '/signup': (context) => const SignUpScreen(),
        '/forgot-password': (context) => const ForgotPasswordScreen(),
        '/reset-password': (context) => const ResetPasswordScreen(),
        '/home': (context) {
          final role =
            (ModalRoute.of(context)?.settings.arguments as String?) ??

```

```

        "guest";
        return Shell(role: role, initialIndex: 0);
      },
      '/recipes': (context) {
        final role =
          (ModalRoute.of(context)?.settings.arguments as String?) ??
            "guest";
        return Shell(role: role, initialIndex: 1);
      },
      '/manage': (context) {
        final role =
          (ModalRoute.of(context)?.settings.arguments as String?) ??
            "guest";
        return Shell(role: role, initialIndex: 2);
      },
      '/profile': (context) {
        final role =
          (ModalRoute.of(context)?.settings.arguments as String?) ??
            "guest";
        return Shell(role: role, initialIndex: 3);
      },
    ],
  );
}

```

This code is the main entry point and initialization setup for the Recipe Finder application. It begins by importing Flutter's Material components and Firebase packages, then loading the Firebase configuration from `firebase_options.dart`. In `main()`, `WidgetsFlutterBinding.ensureInitialized()` ensures Flutter is fully ready before any asynchronous work starts, followed by `Firebase.initializeApp()` which connects the app to Firebase using platform-specific settings (Android/iOS/Web). After Firebase is successfully initialized, the application launches by calling `runApp()` with `RecipeFinderApp` as the root widget.

`RecipeFinderApp` builds the application using `MaterialApp`, where debug banners are disabled and both light and dark themes are configured through `AppTheme`. Navigation is managed using named routes, starting from the splash screen ('/') as the initial route. The routing table defines paths for onboarding and authentication screens (sign in, sign up, forgot password, and reset password). For the main modules (`/home`, `/recipes`, `/manage`, `/profile`), routing uses a shared container widget `Shell` and passes two important values: `role` and `initialIndex`. The `role` value is taken from route arguments (defaulting to "guest" when not provided), allowing the app to adjust access based on user type, while `initialIndex` selects which tab or section should be displayed first inside the `Shell`.

Overall, this file ensures Firebase is ready before the UI loads, defines the application theme, and centralizes navigation so each feature screen can be accessed consistently.

5.2 Firebase connection

```
static const FirebaseOptions web = FirebaseOptions(  
  apiKey: 'AIzaSyCkapddA5uW0umhHddx_i1-0qymgFCfP0c',  
  appId: '1:470625001038:web:26c62a7ab40eb078b5ed1f',  
  messagingSenderId: '470625001038',  
  projectId: 'recipe-finder-db710',  
  authDomain: 'recipe-finder-db710.firebaseio.com',  
  storageBucket: 'recipe-finder-db710.firebaseio.com',  
  measurementId: 'G-0ZTZ8KJJFC',  
);
```

This code defines the Firebase web configuration for the Recipe Finder application using `FirebaseOptions`, which contains the required identifiers to connect the web app to the correct Firebase project. The `apiKey` links requests from the application to Firebase, while the `appId` uniquely identifies the web app instance under the project. The `projectId` specifies the exact Firebase project where services such as Firestore, Authentication, and Storage are hosted, and `authDomain` supports web authentication processes like sign-in redirects. The `storageBucket` indicates where uploaded files such as recipe images are stored in Firebase Storage, `messagingSenderId` is used for Firebase Cloud Messaging if notifications are enabled, and `measurementId` supports Google Analytics tracking for usage monitoring. Overall, these settings ensure the web version of the application can access Firebase services reliably and securely under the correct project environment.

6. Testing Strategy

Testing is conducted using a combination of automated testing and manual validation to ensure the application functions correctly across devices and user roles (guest vs logged-in user). The strategy focuses on verifying core features such as browsing recipes without login, authentication, bookmarking, recipe management, and Firebase integration, while also ensuring good performance and secure access control.

6.1 Unit Testing (Logic Validation)

Unit tests are used to validate non-UI logic such as input validation, data parsing, and service/repository methods. Examples include verifying recipe model serialization/deserialization, checking search/filter logic, and validating form fields (required title, ingredients format, step ordering). Mocked Firebase services are used where necessary to avoid depending on live databases.

6.2 Widget Testing (UI Component Verification)

Widget tests ensure UI components render correctly and respond to user actions. This includes testing screen layouts, button taps, dialogs (add/edit recipe), form validation messages, and navigation between screens. Key UI flows such as opening recipe details, adding bookmarks, and switching tabs in the Shell navigation are verified.

6.3 Integration Testing (End-to-End User Flow)

Integration tests simulate complete scenarios across multiple screens and services. Typical flows include: guest browsing → viewing details → attempting restricted actions; user sign in → add recipe → upload image → view recipe → bookmark → remove bookmark; and profile actions such as logout. These tests verify Firebase Authentication + Firestore + Storage interactions work together as expected.

6.4 Manual Testing (Device and Usability Testing)

Manual testing is performed on real devices and emulators to confirm responsive UI behavior, correct rendering across screen sizes, and smooth navigation. Testing includes offline/weak network scenarios, loading indicators, error messages, and edge cases like empty search results or missing images. Guest access is validated to ensure browsing works without login while management features remain restricted.

6.5 Security and Role-Based Access Testing (Firebase Rules Validation)

Firebase Security Rules are tested to ensure only authorized actions are allowed. Guests are tested for read-only access, authenticated users are tested for writing and editing only their own recipes, and admin-only actions (if implemented) are verified. Attempts to bypass access control are checked by calling Firestore operations without proper authentication.

6.7 Performance and Stability Testing

Performance checks focus on list loading speed, image loading, Firestore query efficiency, and overall app responsiveness. Large recipe lists are tested to ensure scrolling remains smooth and pagination/limit queries work properly. Crash scenarios are monitored during testing, and optimizations are applied where bottlenecks are found.

This strategy ensures Recipe Finder is functional, secure, and stable, while providing a consistent experience across platforms and user roles.

7. Technical Challenges and Solutions

7.1 Guest Access With Restricted Features

- **Problem:** Recipe browsing must be available without login, but actions like bookmarking, uploading recipes, and managing content must be restricted to authenticated users.
- **Solution:** Implemented role-based routing (default role = “guest”) and enforced access control using Firebase Authentication checks and Firestore Security Rules (read-only for guests, write permissions only for authenticated users and owners).
- **Outcome:** Smooth guest experience while maintaining secure control of protected features.

7.2 Firestore Data Structure and Query Limitations

- **Problem:** Search and filtering can become inefficient if recipe data is not structured correctly, especially when handling keyword search, tags, and categories.
- **Solution:** Designed Firestore documents with searchable fields (title, tags, category) and used indexed queries where applicable (e.g., where filters on category/tags). For keyword search, implemented simplified client-side matching on a limited dataset or stored normalized keywords/tokens for faster filtering.
- **Outcome:** Faster browsing and filtering with scalable database structure.

7.3 Image Upload and Retrieval Handling

- **Problem:** Uploading recipe images requires coordination between Firebase Storage (file upload) and Firestore (saving the image URL), which can fail midway and cause inconsistent data.
- **Solution:** Implemented a two-step upload flow: upload image to Firebase Storage first, retrieve the download URL, then save the recipe document in Firestore with the URL. Added error handling to prevent incomplete recipe records from being stored.
- **Outcome:** Reliable image upload process and consistent recipe data.

7.4 Navigation and Screen Access Control

- **Problem:** Multiple routes and module navigation (Home, Recipes, Manage, Profile) can cause inconsistent access if role and session state are not handled centrally.
- **Solution:** Centralized navigation using a Shell container with `initialIndex` and role-based rendering. Route arguments determine the active module and role, while authentication state controls which actions appear in each module.
- **Outcome:** Consistent navigation flow and reduced UI duplication.

7.5 Offline and Weak Network Behavior

- **Problem:** Mobile users may experience slow or unstable internet connections, causing delays in loading recipes and images.
- **Solution:** Used loading states (progress indicators), graceful error messages, and Firestore caching where suitable. Implemented retry-friendly UI patterns for data refresh.
- **Outcome:** Better usability under poor network conditions and fewer app interruptions.

8. Conclusion

This technical report presented the development of Recipe Finder using Flutter and Firebase, focusing on the selected methodology, system workflow, core implementation design, testing strategy, and key technical challenges with corresponding solutions. The reported approaches ensured the application supported guest browsing while maintaining secure access for authenticated features through controlled routing and Firebase Security Rules. The modular project structure, cloud-first architecture, and phased implementation improved maintainability and scalability, while structured testing (unit, widget, integration, and manual validation) improved reliability across devices and user roles. Overall, the documented decisions and solutions demonstrate that Recipe Finder was implemented in a systematic and technically sound manner, meeting the project objectives for an accessible, organized, and user-friendly recipe management platform.